Experiment No: 02

Title: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Aim: To design and implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms

Objectives:

- To understand parallel bubble sort and merge sort algorithms.
- To implement Bubble Sort and Merge sort algorithms.
- To measure the performance sequential and parallel.

Software and Hardware Requirements

1. **Hardware**
   - Intel i5 /AMD Multi-core Processor
   - Minimum 4/8 GB RAM
2. **Software**
   - Operating System: Linux / Ubuntu / Windows with WSL
   - Compiler: GCC / G++ with OpenMP support
   - Programming Language: C/C++
   - Libraries: OpenMP

Theory

Bubble Sort

Bubble Sort is a simple sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order. It is called "bubble" sort because the algorithm moves the larger elements towards the end of the array in a manner that resembles the rising of bubbles in a liquid.

The basic algorithm of Bubble Sort is as follows:

1. Start at the beginning of the array.

2. Compare the first two elements. If the first element is greater than the second element, swap them.

3. Move to the next pair of elements and repeat step 2.

4. Continue the process until the end of the array is reached.

5. If any swaps were made in step 2-4, repeat the process from step 1.

The time complexity of Bubble Sort is O(n^2), which makes it inefficient for large lists. However, it has the advantage of being easy to understand and implement, and it is useful for educational purposes and for sorting small datasets.

Bubble Sort has limited practical use in modern software development due to its inefficient time complexity of O(n^2) which makes it unsuitable for sorting large datasets. However, Bubble Sort has some advantages and use cases that make it a valuable algorithm to understand, such as:

1. Simplicity: Bubble Sort is one of the simplest sorting algorithms, and it is easy to understand and implement. It can be used to introduce the concept of sorting to beginners and as a basis for more complex sorting algorithms.

2. Small datasets: For very small datasets, Bubble Sort can be an efficient sorting algorithm, as its overhead is relatively low.

3. Partially sorted datasets: If a dataset is already partially sorted, Bubble Sort can be very efficient. Since Bubble Sort only swaps adjacent elements that are in the wrong order, it has a low number of operations for a partially sorted dataset.

4. Performance optimization: Although Bubble Sort itself is not suitable for sorting large datasets, some of its techniques can be used in combination with other sorting algorithms to optimize their performance. For example, Bubble Sort can be used to optimize the performance of Insertion Sort by reducing the number of comparisons needed.

How Parallel Bubble Sort Work?

● Parallel Bubble Sort is a modification of the classic Bubble Sort algorithm that takes advantage of parallel processing to speed up the sorting process.

● In parallel Bubble Sort, the list of elements is divided into multiple sublists that are sorted concurrently by multiple threads. Each thread sorts its sublist using the regular Bubble Sort algorithm. When all sublists have been sorted, they are merged together to form the final sorted list.

● The parallelization of the algorithm is achieved using OpenMP, a programming API that supports parallel processing in C++, Fortran, and other programming languages. OpenMP provides a set of compiler directives that allow developers to specify which parts of the code can be executed in parallel.

● In the parallel Bubble Sort algorithm, the main loop that iterates over the list of elements is

divided into multiple iterations that are executed concurrently by multiple threads. Each thread sorts a subset of the list, and the threads synchronize their work at the end of each iteration to ensure that the elements are properly ordered.

● Parallel Bubble Sort can provide a significant speedup over the regular Bubble Sort algorithm, especially when sorting large datasets on multi-core processors. However, the speedup is limited by the overhead of thread creation and synchronization, and it may not be worth the effort for small datasets or when using a single-core processor.

How to measure the performance of sequential and parallel algorithms?

To measure the performance of sequential Bubble sort and parallel Bubble sort algorithms, you can follow these steps:

1. Implement both the sequential and parallel Bubble sort algorithms.
2. Choose a range of test cases, such as arrays of different sizes and different degrees of sortedness, to test the performance of both algorithms.
3. Use a reliable timer to measure the execution time of each algorithm on each test case.
4. Record the execution times and analyze the results.

When measuring the performance of the parallel Bubble sort algorithm, you will need to specify the number of threads to use. You can experiment with different numbers of threads to find the optimal value for your system.

Performance Analysis of sequential & parallel Bubble sort

1. Use **same dataset** for both algorithms.
2. Run on **same machine**.
3. Use **multiple runs** and take the average.

Input Size (N)	Sequential Time	Parallel Time
10,000	**-- ms**	**-- ms**
10,000	**-- ms**	**-- ms**
1,00,000	**-- ms**	**-- ms**
10,00,000	**-- ms**	**-- ms**

Note: Here students will execute their codes and note down the respective time required for execution in milliseconds in above table.

Merge Sort

Merge sort is a sorting algorithm that uses a divide-and-conquer approach to sort an array or a list of elements. The algorithm works by recursively dividing the input array into two halves, sorting each half, and then merging the sorted halves to produce a sorted output.

The merge sort algorithm can be broken down into the following steps:

1. Divide the input array into two halves.
2. Recursively sort the left half of the array.
3. Recursively sort the right half of the array.
4. Merge the two sorted halves into a single sorted output array.

● The merging step is where the bulk of the work happens in merge sort. The algorithm compares the first elements of each sorted half, selects the smaller element, and appends it to the output array. This process continues until all elements from both halves have been appended to the output array.

● The time complexity of merge sort is O(n log n), which makes it an efficient sorting algorithm for large input arrays. However, merge sort also requires additional memory to store the output array, which can make it less suitable for use with limited memory resources.

● In simple terms, we can say that the process of merge sort is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

● One thing that you might wonder is what is the specialty of this algorithm. We already have a number of sorting algorithms then why do we need this algorithm? One of the main advantages of merge sort is that it has a time complexity of O(n log n), which means it can sort large arrays relatively quickly. It is also a stable sort, which means that the order of elements with equal values is preserved during the sort.

● Merge sort is a popular choice for sorting large datasets because it is relatively efficient and easy to implement. It is often used in conjunction with other algorithms, such as quicksort, to improve the overall performance of a sorting routine.

How Parallel Merge Sort Work?

● Parallel merge sort is a parallelized version of the merge sort algorithm that takes advantage of multiple processors or cores to improve its performance. In parallel merge sort, the input array is divided into smaller subarrays, which are sorted in parallel using multiple processors

or cores. The sorted subarrays are then merged together in parallel to produce the final sorted output.

The parallel merge sort algorithm can be broken down into the following steps:

● Divide the input array into smaller subarrays.
● Assign each subarray to a separate processor or core for sorting.
● Sort each subarray in parallel using the merge sort algorithm.
● Merge the sorted subarrays together in parallel to produce the final sorted output.
● The merging step in parallel merge sort is performed in a similar way to the merging step in the sequential merge sort algorithm. However, because the subarrays are sorted in parallel, the merging step can also be performed in parallel using multiple processors or cores. This can significantly reduce the time required to merge the sorted subarrays and produce the final output.
● Parallel merge sort can provide significant performance benefits for large input arrays with many elements, especially when running on hardware with multiple processors or cores. However, it also requires additional overhead to manage the parallelization, and may not always provide performance improvements for smaller input sizes or when run on hardware with limited parallel processing capabilities.

To measure the performance of sequential and parallel merge sort algorithms, you can perform experiments on different input sizes and numbers of processors or cores. By measuring the execution time, speedup, efficiency, and scalability of the algorithms under different conditions, you can determine which algorithm is more efficient for different input sizes and hardware configurations. Additionally, you can use profiling tools to analyze the performance of the algorithms and identify areas for optimization

Performance Analysis of sequential & parallel Merge sort

1. Use **same dataset** for both algorithms.
2. Run on **same machine**.
3. Use **multiple runs** and take the average.

Input Size (N)	Sequential Time	Parallel Time
1000	**-- ms**	**-- ms**
10,000	**-- ms**	**-- ms**
1,00,000	**-- ms**	**-- ms**
10,00,000	**-- ms**	**-- ms**

Programming Task: Write a C++ program to implement parallel Bubble sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Note: Students will write program using OpenMP and take the print of code and output. Attach these prints (1. Code 2. Output) at the end of write-up. (Don't write the code by hands.)

Conclusion:

Assignment Questions: (Solve the questions below)

1. What are the advantages of Parallel Bubble Sort?
2. Difference between serial bubble sort and parallel bubble sort
3. How do you implement Parallel MergeSort using OpenMP?
4. What are the advantages of Parallel MergeSort?

Experiment No: 01

Title: Design and implement Parallel Breadth First Search (BFS) and Depth First Search (DFS) based on existing algorithms using OpenMP. Use a Tree or an Undirected Graph for BFS and DFS.

Aim: To design and implement parallel Breadth First Search (BFS) and Depth First Search (DFS) algorithms using OpenMP and analyze their execution on a tree or undirected graph structure.

Objectives:

- To understand graph traversal algorithms.
- To implement BFS and DFS algorithms.
- To parallelize graph traversal using OpenMP directives.
- To compare sequential and parallel traversal performance.

Software and Hardware Requirements

1. **Hardware**
    - Intel/AMD Multi-core Processor
    - Minimum 4/8 GB RAM
2. **Software**
    - Operating System: Linux / Ubuntu / Windows with WSL
    - Compiler: GCC with OpenMP support
    - Programming Language: C/C++
    - Libraries: OpenMP

Theory

Graph Traversal

Graph traversal is the process of visiting all vertices of a graph systematically.

Two common traversal techniques are:

- **Breadth First Search (BFS)**
-
- **Depth First Search (DFS)**

These algorithms are widely used in Artificial Intelligence, Network routing, Path finding, Social network analysis, Web crawling etc.

Breadth First Search (BFS)

BFS is a fundamental graph traversal algorithm that explores nodes level-by-level, visiting all neighbors at the current depth before moving to nodes at the next depth level. It uses a queue data structure to manage traversal, making it ideal for finding the shortest path in unweighted graphs, typically in ***O (V+E)*** time.

How BFS Works?

Step 1. Start from the source node: Choose a starting node to begin traversal.

Step 2. Mark the source as visited: Keep a record of visited nodes to avoid repetition or loops.

Step 3. Enqueue the source node: Use a queue (FIFO structure) to store nodes to be explored.

Step 4. Process the queue: BFS dequeues a node, visits its unvisited neighbors, marks them visited, and enqueues them.

Step 5. Move level by level: Continue until the queue is empty, visiting all nodes layer by layer.

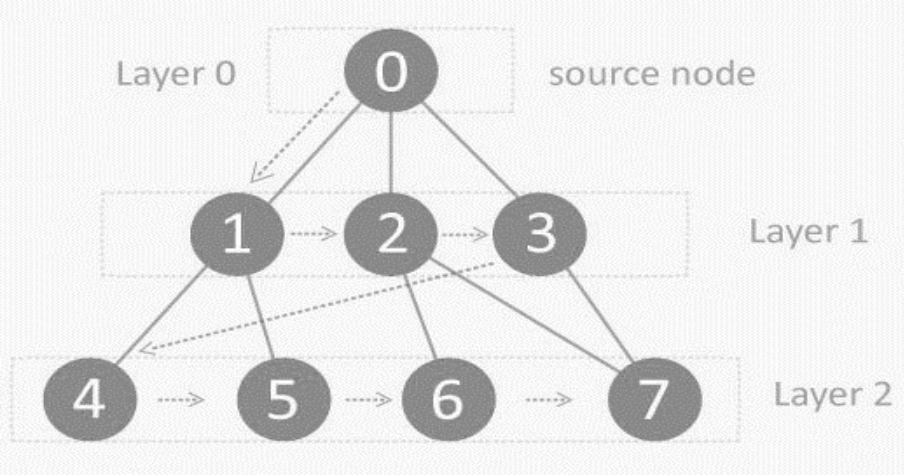

Fig. 1 Graph with 8 nodes.

Considering above graph it start traversing from 0 and visit its child nodes 1, 2, and 3. Store them in the order in which they are visited. This will allow you to visit the child nodes of 1 first (i.e. 4 and 5), then of 2 (i.e. 6 and 7), and then of 3 (i.e. 7).

Concept of OpenMP

● OpenMP (Open Multi-Processing) is an application programming interface (API) that supports shared-memory parallel programming in C, C++, and Fortran. It is used to write parallel programs that can run on multicore processors, multiprocessor systems, and parallel computing clusters.

● OpenMP provides a set of directives and functions that can be inserted into the source code of a program to parallelize its execution. These directives are simple and easy to use, and they can be applied to loops, sections, functions, and other program constructs. The compiler then generates parallel code that can run on multiple processors concurrently.

● OpenMP programs are designed to take advantage of the shared-memory architecture of modern processors, where multiple processor cores can access the same memory. OpenMP uses a fork-join model of parallel execution, where a master thread forks multiple worker threads to execute a parallel region of the code, and then waits for all threads to complete before continuing with the sequential part of the code.

● OpenMP is widely used in scientific computing, engineering, and other fields that require high-performance computing. It is supported by most modern compilers and is available on a wide range of platforms, including desktops, servers, and supercomputers.

How Parallel BFS Work

● Parallel BFS (Breadth-First Search) is an algorithm used to explore all the nodes of a graph or tree systematically in parallel. It is a popular parallel algorithm used for graph traversal in distributed computing, shared-memory systems, and parallel clusters.

● The parallel BFS algorithm starts by selecting a root node or a specified starting point, and then assigning it to a thread or processor in the system. Each thread maintains a local queue of nodes to be visited and marks each visited node to avoid processing it again.

● The algorithm then proceeds in levels, where each level represents a set of nodes that are at a certain distance from the root node. Each thread processes the nodes in its local queue at the current level, and then exchanges the nodes that are adjacent to the current level with other threads or processors. This is done to ensure that the nodes at the next level are visited by the next iteration of the algorithm.

● The parallel BFS algorithm uses two phases: the computation phase and the communication phase. In the computation phase, each thread processes the nodes in its local queue, while in the communication phase, the threads exchange the nodes that are adjacent to the current level with other threads or processors.

● The parallel BFS algorithm terminates when all nodes have been visited or when a specified node has been found. The result of the algorithm is the set of visited nodes or the shortest path from the root node to the target node.

● Parallel BFS can be implemented using different parallel programming models, such as OpenMP, MPI, CUDA, and others. The performance of the algorithm depends on the number of threads or processors used, the size of the graph, and the communication overhead between the threads or processors.

Depth First Search (DFS)

The DFS algorithm is a recursive algorithm that uses the idea of backtracking. It involves exhaustive searches of all the nodes by going ahead, if possible, else by backtracking.

DFS can be implemented using stacks. The basic idea is as follows:

1. Pick a starting node and push all its adjacent nodes into a stack.
2. Pop a node from stack to select the next node to visit and push all its adjacent nodes into a stack.
3. Repeat this process until the stack is empty. However, ensure that the nodes that are visited are marked. This will prevent you from visiting the same node more than once. If you do not mark the nodes that are visited and you visit the same node more than once, you may end up in an infinite loop.

Considering above graph in Fig. 1, it start traversing from 0 and visit its child nodes 1, 4, and 5. Then backtrack to 0 and next child of 0 which is 2 then 6 and backtrack again and next child of 2 which is 7. So the DFS sequence will be 0,1,4,5,2,6,7.

How Parallel DFS Work?

● Parallel Depth-First Search (DFS) is an algorithm that explores the depth of a graph structure to search for nodes. In contrast to a serial DFS algorithm that explores nodes in a sequential manner, parallel DFS algorithms explore nodes in a parallel manner, providing a significant speedup in large graphs.

● Parallel DFS works by dividing the graph into smaller subgraphs that are explored simultaneously. Each processor or thread is assigned a subgraph to explore, and they work independently to explore the subgraph using the standard DFS algorithm. During the exploration process, the nodes are marked as visited to avoid revisiting them.

● To explore the subgraph, the processors maintain a stack data structure that stores the nodes in the order of exploration. The top node is picked and explored, and its adjacent nodes are pushed onto the stack for further exploration. The stack is updated concurrently by the processors as they explore their subgraphs.

● Parallel DFS can be implemented using several parallel programming models such as OpenMP, MPI, and CUDA. In OpenMP, the #pragma omp parallel for directive is used to distribute the work among multiple threads. By using this directive, each thread operates on a different part of the graph, which increases the performance of the DFS algorithm.

Programming Task: Write a C++ program to implement parallel BFS and DFS based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .

Note: Students will write program using OpenMP and take the print of code and output. Attach these prints (1. Code 2. Output) at the end of write-up. (<u>Don't write the code by hands.)</u>

Conclusion:

Assignment Questions: (Solve the questions below)

1. What is OpenMP? What is its significance in parallel programming?
2. Write Down Commands used in OpenMP?
3. How can BFS be parallelized using OpenMP? Describe the parallel BFS algorithm using OpenMP.
4. What is the advantage of using parallel programming in DFS?

Experiment No: 04

Title: Write a CUDA Program for:

1. Addition of two large vectors
2. Matrix Multiplication using CUDA C

Aim: To design and implement GPU-based parallel programs using CUDA C for:

- Vector addition of large datasets
- Matrix multiplication using CUDA parallel threads

Objectives:

- To understand the CUDA programming model.
- To implement vector addition using GPU parallel threads.
- To implement matrix multiplication using CUDA kernels.

Software and Hardware Requirements

Hardware

- NVIDIA GPU with CUDA support
- Minimum 8 GB RAM
- Multi-core CPU

Software

- Operating System: Linux / Ubuntu / Windows
- CUDA Toolkit
- NVIDIA GPU Driver
- Programming Language: CUDA C/C++

Theory

GPU Computing

Graphics Processing Units (GPUs) contain thousands of cores capable of performing parallel operations. CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by NVIDIA that allows programmers to use GPUs for general-purpose computation.

What is CUDA?

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It allows developers to use the power of NVIDIA graphics processing units (GPUs) to accelerate computation tasks in various applications, including scientific computing, machine learning, and computer vision. CUDA provides a set of programming APIs, libraries, and tools that enable developers to write and execute parallel code on NVIDIA GPUs. It supports popular programming languages like C, C++, and Python, and provides a simple programming model that abstracts away much of the low-level details of GPU architecture. Using CUDA, developers can exploit the massive parallelism and high computational power of GPUs to accelerate computationally intensive tasks, such as matrix operations, image processing, and deep learning. CUDA has become an important tool for scientific research and is widely used in fields like physics, chemistry, biology, and engineering.

CUDA Programming Model

CUDA programs consist of:

Host

- CPU
- Controls program execution

Device

- GPU
- Executes parallel kernels

Steps for Addition of two large vectors using CUDA

1. Define the size of the vectors: In this step, you need to define the size of the vectors that you want to add. This will determine the number of threads and blocks you will need to use to parallelize the addition operation.

2. Allocate memory on the host: In this step, you need to allocate memory on the host for the two vectors that you want to add and for the result vector. You can use the C malloc function to allocate memory.

3. Initialize the vectors: In this step, you need to initialize the two vectors that you want to add on the host. You can use a loop to fill the vectors with data.

4. Allocate memory on the device: In this step, you need to allocate memory on the device for the two vectors that you want to add and for the result vector. You can use the CUDA function cudaMalloc to allocate memory.

5. Copy the input vectors from host to device: In this step, you need to copy the two input vectors from the host to the device memory. You can use the CUDA function cudaMemcpy to copy the vectors.

6. Launch the kernel: In this step, you need to launch the CUDA kernel that will perform the addition operation. The kernel will be executed by multiple threads in parallel. You can use the <<<...>>> syntax to specify the number of blocks and threads to use.

7. Copy the result vector from device to host: In this step, you need to copy the result vector from the device memory to the host memory. You can use the CUDA function cudaMemcpy to copy the result vector.

8. Free memory on the device: In this step, you need to free the memory that was allocated on the device. You can use the CUDA function cudaFree to free the memory.

9. **Free memory on the host:** In this step, you need to free the memory that was allocated on the host. You can use the C free function to free the memory

Execution of Program over CUDA Environment

Here are the steps to run a CUDA program for adding two large vectors:

1. Install CUDA Toolkit: First, you need to install the CUDA Toolkit on your system. You can download the CUDA Toolkit from the NVIDIA website and follow the installation instructions provided.

2. Set up CUDA environment: Once the CUDA Toolkit is installed, you need to set up the CUDA environment on your system. This involves setting the PATH and LD_LIBRARY_PATH environment variables to the appropriate directories.

3. Write the CUDA program: You need to write a CUDA program that performs the addition of two large vectors. You can use a text editor to write the program and save it with a .cu extension.

4. Compile the CUDA program: You need to compile the CUDA program using the nvcc compiler that comes with the CUDA Toolkit. The command to compile the program is:

```
nvcc -o program_name program_name.cu
```

5. Run the CUDA program: Finally, you can run the CUDA program by executing the executable file generated in the previous step. The command to run the program is:

```
./program_name
```

This will execute the program and perform the addition of two large vectors.

Steps for Matrix Multiplication using CUDA

Here are the steps for implementing matrix multiplication using CUDA C:

1. Matrix Initialization: The first step is to initialize the matrices that you want to multiply. You can use standard C or CUDA functions to allocate memory for the matrices and initialize their values. The matrices are usually represented as 2D arrays.

2. Memory Allocation: The next step is to allocate memory on the host and the device for the matrices. You can use the standard C malloc function to allocate memory on the host and the CUDA function cudaMalloc() to allocate memory on the device.

3. Data Transfer: The third step is to transfer data between the host and the device. You can use the CUDA function cudaMemcpy() to transfer data from the host to the device or vice versa.

4. Kernel Launch: The fourth step is to launch the CUDA kernel that will perform the matrix multiplication on the device. You can use the <<<...>>> syntax to specify the number of blocks and threads to use. Each thread in the kernel will compute one element of the output matrix.

5. Device Synchronization: The fifth step is to synchronize the device to ensure that all kernel executions have completed before proceeding. You can use the CUDA function **cudaDeviceSynchronize()** to synchronize the device.

6. Data Retrieval: The sixth step is to retrieve the result of the computation from the device to the host. You can use the CUDA function cudaMemcpy() to transfer data from the device to the host.

7. Memory Deallocation: The final step is to deallocate the memory that was allocated on the host and the device. You can use the C free function to deallocate memory on the host and the CUDA function cudaFree() to deallocate memory on the device.

Execution of Program over CUDA Environment

1. Install CUDA Toolkit: First, you need to install the CUDA Toolkit on your system. You can download the CUDA Toolkit from the NVIDIA website and follow the installation instructions provided.

2. Set up CUDA environment: Once the CUDA Toolkit is installed, you need to set up the CUDA environment on your system. This involves setting the PATH and LD_LIBRARY_PATH environment variables to the appropriate directories.

3. Write the CUDA program: You need to write a CUDA program that performs the addition of two large vectors. You can use a text editor to write the program and save it with a .cu extension.

4. Compile the CUDA program: You need to compile the CUDA program using the nvcc compiler that comes with the CUDA Toolkit. The command to compile the program is:

```
nvcc -o program_name program_name.cu
```

5. Run the CUDA program: Finally, you can run the CUDA program by executing the executable file generated in the previous step. The command to run the program is:

```
./program_name
```

Programming Task: Write a CUDA Program for: 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C

Note: Students will write program using OpenMP and take the print of code and output. Attach these prints (1. Code 2. Output) at the end of write-up. (Don't write the code by hands.)

Conclusion:

Assignment Questions: (Solve the questions below)

1. What is the purpose of using CUDA to perform addition of two large vectors?
2. How do you allocate memory for the vectors on the device using CUDA?
3. What are the advantages of using CUDA to perform matrix multiplication compared to using a CPU?
4. How do you handle matrices that are too large to fit in GPU memory in CUDA matrix multiplication?

Experiment No: 03

Title: Implement Min, Max, Sum and Average operations using Parallel Reduction.

Aim: To design and implement parallel reduction operations for computing Minimum, Maximum, Sum and Average of a set of numbers using OpenMP.

Objectives:

- To understand the concept of parallel reduction.
- To implement parallel computation using OpenMP directives.
- To compute minimum, maximum, sum, and average values of an array in parallel.

Software and Hardware Requirements

1. **Hardware**
   - Intel i5 /AMD Multi-core Processor
   - Minimum 4/8 GB RAM
2. **Software**
   - Operating System: Linux / Ubuntu / Windows with WSL
   - Compiler: GCC / G++ with OpenMP support
   - Programming Language: C/C++
   - Libraries: OpenMP

Theory

Parallel Reduction.

Here's a function-wise manual on how to understand and run the sample C++ program that demonstrates how to implement Min, Max, Sum, and Average operations using parallel reduction.

1. Min_Reduction function

- The function takes in a vector of integers as input and finds the minimum value in the vector using parallel reduction.
- The OpenMP reduction clause is used with the "min" operator to find the minimum value across all threads.

- The minimum value found by each thread is reduced to the overall minimum value of the entire array.
- The final minimum value is printed to the console.

2. Max_Reduction function

- The function takes in a vector of integers as input and finds the maximum value in the vector using parallel reduction.
- The OpenMP reduction clause is used with the "max" operator to find the maximum value across all threads.
- The maximum value found by each thread is reduced to the overall maximum value of the entire array.
- The final maximum value is printed to the console.

3. Sum_Reduction function

- The function takes in a vector of integers as input and finds the sum of all the values in the vector using parallel reduction.
- The OpenMP reduction clause is used with the "+" operator to find the sum across all threads.
- The sum found by each thread is reduced to the overall sum of the entire array.
- The final sum is printed to the console.

4. Average_Reduction function

- The function takes in a vector of integers as input and finds the average of all the values in the vector using parallel reduction.
- The OpenMP reduction clause is used with the "+" operator to find the sum across all threads.
- The sum found by each thread is reduced to the overall sum of the entire array.
- The final sum is divided by the size of the array to find the average.
- The final average value is printed to the console.

Programming Task: Write a C++ program to implement parallel Bubble sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Procedure to implement above task:

Main Function

- The function initializes a vector of integers with some values.
- The function calls the min_reduction, max_reduction, sum_reduction, and average_reduction functions on the input vector to find the corresponding values.
- The final minimum, maximum, sum, and average values are printed to the console.

6. Compiling and running the program

Compile the program: You need to use a C++ compiler that supports OpenMP, such as g++ or clang. Open a terminal and navigate to the directory where your program is saved. Then, compile the program using the following command:

$ g++ -fopenmp program.cpp -o program

This command compiles your program and creates an executable file named "program". The "-fopenmp" flag tells the compiler to enable OpenMP.

Run the program: To run the program, simply type the name of the executable file in the terminal and press Enter:

$ ./program

Note: Students will write program using OpenMP and take the print of code and output. Attach these prints (1. Code 2. Output) at the end of write-up. (Don't write the code by hands.)

Conclusion:

Assignment Questions: (Solve the questions below)

1 What are the benefits of using parallel reduction for basic operations on large arrays?

2. How does OpenMP's "reduction" clause work in parallel reduction?

3. How do you set up a C++ program for parallel computation with OpenMP?

4. What are the performance characteristics of parallel reduction, and how do they vary based on input size?